Linker Scripts for ld01: Design

mc6801-tools

November 2026

Contents

1 Summary **2**

2 Background: how ld01 links today **3**

2.1 Input . . . 3

2.2 Module selection . . . 3

2.3 Layout . . . 3

2.4 Output . . . 4

2.5 Limitations . . . 4

3 Goals and non-goals **4**

3.1 Goals . . . 4

3.2 Non-goals . . . 5

4 Concepts **5**

5 The script language **5**

5.1 Commands . . . 5

5.2 MEMORY . . . 6

5.3 SECTIONS . . . 6

5.4 Input section patterns . . . 7

5.5 Expressions . . . 7

5.6 Symbols . . . 8

6 Module selection **8**

6.1 Orphans . . . 9

7 Output formats **9**

8 Command line **9**

8.1 Printing the default script 10

9 The default script **10**

10 Examples **12**

10.1 A RAM program for the MC-10 12

10.2 A ROM program 13

10.3 Fixed addresses 14

11 Error checking **14**

12 Implementation **14**

12.1 1. Per-module placement 15

12.2 2. Script parsing 15

12.3 3. Layout 15

12.4 4. Output 15

12.5 5. Documentation 16

13 Testing **16**

14 Phases **16**

15 Open questions **17**

1 Summary

This document describes how `ld01` could support *linker scripts*: text files that control how a program is laid out in memory. A script can:

- say which modules, and which parts of them, go into each output section, and in what order;
- give the start address of each section, or place sections into named memory regions such as ROM and RAM;
- load a section at one address while it runs at another, for data that is copied from ROM to RAM at startup;
- define symbols that no object file defines, such as the top of the stack or the address of an I/O port;
- fill gaps, align sections, force modules into the link, and choose the output format.

`ld01 -P` prints the default script, as a starting point for writing your own.

The design keeps today's behavior as the default. The current fixed layout is expressed as a built-in *default script*, so a link without a script produces byte-identical output to today's `ld01`. The script language is a subset of the GNU `ld` script language, adapted to the input that `as01` produces.

2 Background: how ld01 links today

2.1 Input

`as01` produces object files in the Introl format. Each module has 16 *location counters* (also called segments), numbered 0 to 15. The assembler selects them with `LOC n`, or with `.TEXT` (counter 0), `.DATA`, `.ROM` and `.BSS` (all counter 3). For each counter, a module has:

- a *data part*: bytes, relocation records and *skip* records. `RMB` and similar directives produce skip records, which `ld01` writes out as zero bytes. So reserved space takes up room in the output.
- a *common part*: common blocks (`COMM`, `LCOMM`), which take up address space but have no contents.

Symbols are either absolute, or relative to one of the module's location counters. An `IMPORT` also records the location counter that was current when it was declared. If the defining module puts the symbol in a different counter, or makes it absolute, `ld01` warns ("redefined with different segment or relocatability") and leaves the symbol undefined. So data symbols have to be imported while the data counter is current.

2.2 Module selection

`ld01` reads every object file and archive named on the command line, then loads only the modules it can reach by following symbol references (`linksyms.c`). It starts from the modules that define `ENTRY` symbols, `_main` and `start`. A module that nothing refers to is left out, even if it was named on the command line.

The program's entry point is the first `ENTRY` symbol. If there is none, `_main` becomes the entry point.

2.3 Layout

`write_elks()` in `writex86.c` uses a fixed layout:

Counters	Placement
0, 1	Text, from the `-T` address, in that order
2	Start of data, moved up to the next 256-byte page if it would cross a page boundary
3–15	Data, after counter 2, in order

Within each counter, the modules' data parts are concatenated in command-line order. Each module's part is padded to a multiple of 4 bytes, a leftover from the 386 version. Each counter's common part follows its data part, before the next counter starts. `-D` moves the data segment to a separate address. With `-i` (separate instruction and data spaces), data has its own address space starting at the `-D` address, or 0. `-H` sets `__heap_top`.

Addresses are assigned in two passes:

1. While summing sizes, each symbol's value becomes its offset within its counter across all modules (`segsz[]`), and the counter's start address (`segbase[]`) is added afterwards.
2. While writing, `linkmod()` reads each module's records. Offset relocations use the running `segbase[seg]` for the current module, and symbol relocations use the symbol's final value. `segadj[seg]` gives each counter's position in the output file.

2.4 Output

The output is an a.out file: a 32-byte header with text, data and bss sizes and the entry point, the text and data, and a symbol table. With `-d`, it's a headerless memory image of text followed by data. `objcopy01` turns a.out files into S-records, Intel hex and cassette images.

2.5 Limitations

The fixed layout can't express common embedded layouts. For example:

- **ROM programs.** Code at `$E000`, the reset and interrupt vectors at `$FFF0`, variables in the internal RAM at `$0080`, and initialized data copied from ROM to RAM. Today, the vector table needs an `ORG` in an absolute program, and there's no way to load data in one place and run it in another.
- **Placing a module at a fixed address**, for example a jump table that other programs call at a known address.
- **Ordering** modules other than by command-line order.
- **Defining addresses** such as the stack top or memory-mapped I/O without writing a module just for them.
- **Including a module** that nothing refers to, such as an interrupt vector table.

3 Goals and non-goals

3.1 Goals

1. **Compatibility.** Without a script, `ld01` behaves exactly as today, and its output is byte-identical.
2. **Familiar syntax.** The script language follows GNU `ld`, so the GNU documentation and existing knowledge mostly apply.
3. **Precise control.** Scripts control which input goes where, the order, the addresses, alignment, fill, and symbols.

4. **Clear errors.** Overlapping sections, regions that overflow, addresses above `$FFFF` and undefined symbols are reported with the script's file and line number.
5. **Multi-region output.** Programs that occupy several separate areas of memory can be written as S-records, Intel hex or a filled binary image.

3.2 Non-goals

- Full GNU `ld` script compatibility. Features that make no sense for `as01` objects, such as ELF section flags, `.init_array`, overlays or version scripts, are left out.
- Changes to the object file format.
- Linking 6809 or 8086 objects.

4 Concepts

Input section. One location counter's data part of one module, or its common part. They're named by counter: `LOC0` to `LOC15`, and `COMMON0` to `COMMON15` for the common parts. For convenience, `.text` means `LOC0` and `.data` means `LOC3`, as `as01` uses them. `COMMON` means all common parts.

Output section. A named, contiguous piece of the program, such as `.text` or `.vectors`, made up of input sections and fill. It has a *virtual memory address* (VMA), where it runs, and a *load memory address* (LMA), where it's stored. They're the same unless the script says otherwise.

Memory region. A named range of addresses, such as ROM or RAM. Output sections can be placed into a region instead of at a fixed address. The linker then allocates them in order and checks that they fit.

Location counter (`.`). In a script, `.` is the current address within an output section, or between sections. Assigning to it moves the address, for example to leave a gap.

Loadable section. An output section with contents that are written to the output. A section marked `NOLOAD`, or one that contains only common blocks, takes up address space but isn't written.

5 The script language

A script is a list of commands. Whitespace and comments (`/* ... */` and `#` to the end of the line) are ignored.

5.1 Commands

```
ENTRY(symbol)                   start the program at symbol
OUTPUT_FORMAT(format)           aout (default), binary, srec or ihex
FILL(byte)                      default gap fill for binary output
MEMORY { region-list }          define memory regions
SECTIONS { section-list }       define output sections
symbol = expression ;           define a symbol
PROVIDE(symbol = expression) ;  define it only if needed and not defined
```

```
HIDDEN(symbol = expression) ;   define a symbol for use in the script only
ASSERT(expression, "message")   stop with an error if false
INCLUDE "file"                  read another script
```

Without `ENTRY`, the entry point is chosen as today: the first `ENTRY` symbol in the object files, otherwise `_main`.

5.2 MEMORY

```
MEMORY
{
    ROM (rx) : ORIGIN = 0xE000, LENGTH = 0x1FF0
    VEC (r)  : ORIGIN = 0xFFF0, LENGTH = 16
    RAM (rw) : ORIGIN = 0x0080, LENGTH = 128
}
```

The attributes in parentheses (`r`, `w`, `x`) are recorded for the map and for error messages only. A section placed in a region (`> ROM`) starts at the region's next free address. The linker reports an error if the section doesn't fit.

5.3 SECTIONS

```
SECTIONS
{
    name [address] [(NOLOAD)] : [AT(lma)] [ALIGN(n)] [SUBALIGN(n)]
    {
        statements
    } [> region] [AT> region] [= fill]
    ...
}
```

- `address` sets the VMA. Without it, the section follows the previous one in the same region, or the current value of `.`.
- `AT(lma)` or `AT> region` sets a separate load address.
- `ALIGN(n)` aligns the section's start. `SUBALIGN(n)` pads each input section within it to a multiple of `n` bytes, so each one starts and ends on a multiple of `n`. (GNU `ld` only aligns the start; padding the end as well matches today's `ld01`.) The default is no alignment, except in the default script (see below).
- `NOLOAD` reserves the space without writing anything to the output.
- `= fill` sets the byte used for gaps inside the section. The default is

  0.

Statements inside a section:

```
pattern                          include input sections
KEEP(pattern)                    include them, and load their modules
ONEPAGE { statements }           keep them within one 256-byte page
symbol = expression ;            define a symbol, often using .
PROVIDE(symbol = expression) ;
HIDDEN(symbol = expression) ;
. = expression ;                 move within the section
BYTE(expression) ;               put data into the section
WORD(expression) ;               16 bits, high byte first
FILL(byte) ;                     fill byte for later gaps
```

`ONEPAGE` places the statements inside it together. If their contents would cross a 256-byte page boundary, the block starts at the next page instead, and assignments inside it see the moved addresses. It's an error if the contents are larger than a page. Today's `ld01` treats location counter 2 this way.

5.4 Input section patterns

```
file(sections)
```

`file` is matched against each object file name, with or without its directory. Archive members are named `archive:member`. The wildcards `*` and `?` work in both parts, so `libnum.a:*` means every member of `libnum.a`, and `*` means every file. `sections` is a list of input section names, also with wildcards: `LOC*`, `LOC0 LOC1`, `.text`, `COMMON`.

```
crtstart.o(.text)                counter 0 of crtstart.o
*(.text LOC1)                    counters 0 and 1 of every other module
libnum.a:one.o(LOC*)             all counters of one archive member
*(COMMON)                        all common blocks
```

Each input section is placed by the first pattern that matches it, in script order. A pattern only matches modules that are being loaded, except inside `KEEP`.

5.5 Expressions

Expressions use 16-bit unsigned arithmetic with the C operators `+ - * / % & | ^ ~ << >> !`, the comparisons `== != < <= > >=`, `&& ||`, `? :` and parentheses.

Numbers are written as for the rest of `ld01`, like C: `0x` or `$` for hexadecimal, a leading `0` for octal, otherwise decimal. A `K` suffix multiplies by 1024.

Functions:

Function	Value
`.`	The current address
`ADDR(section)`	VMA of an output section

Function	Value
`LOADADDR(section)`	LMA of an output section
`SIZEOF(section)`	Size of an output section
`ORIGIN(region)`, `LENGTH(region)`	Start and length of a region
`ALIGN(n)`, `ALIGN(expr, n)`	`.` or `expr` rounded up to a multiple of `n`
`DEFINED(symbol)`	1 if the symbol is defined, else 0
`MAX(a, b)`, `MIN(a, b)`	Larger or smaller value

A symbol can be used before it's defined. The linker evaluates assignments in a loop until values stop changing (see Implementation).

5.6 Symbols

A symbol defined in a script is absolute. It satisfies `IMPORT`s in any module, just like an `EXPORT`ed symbol. Unlike a symbol defined in an object file, it matches an import from any location counter: the segment and relocatability check described in Input doesn't apply to script symbols.

- `symbol = expression;` always defines it. An object file defining the same symbol is an error.
- `PROVIDE(symbol = expression);` defines it only if some module refers to it and no module defines it. This is how the default script defines `__btext` and the other linker symbols today.
- `HIDDEN(symbol = expression);` defines a symbol that only the script can use. It doesn't satisfy imports and isn't written to the output's symbol table.

A symbol may be assigned more than once. Assignments take effect in order, as the linker walks through the script, so an expression sees the most recent value. A reference to a symbol that's only assigned later sees the final value from the previous pass.

The linker also defines symbols for each output section: `__name_start`, `__name_end` and `__name_load` (the section's LMA), with any leading dot in the name dropped. For example, `__data_load` is the address in ROM where `.data` is stored. These are provided like `PROVIDE`, so they only appear if something uses them.

6 Module selection

Without a script, `ld01` loads only modules reachable from the entry points. A script changes that in two ways:

- **`KEEP`.** The modules of every input section matched by a `KEEP` pattern are loaded, along with everything they refer to. This is how to include a vector table that nothing calls.
- **Script symbols.** A symbol defined by the script satisfies imports. `ENTRY(symbol)`, and symbols used in `ASSERT`, count as references.

Archive members are still loaded only when needed or kept, as today.

6.1 Orphans

An input section of a loaded module that no pattern matches is an *orphan*. The linker places it where the default script would, after the last output section that holds the same kind of input (`LOC0` and `LOC1` with text, others with data), and prints a warning naming it. The `-Z` option (see below) makes orphans an error instead, for builds that must place everything explicitly.

7 Output formats

A script that places sections in several separate areas can't always be written as an a.out file, which has just a text and a data part. So `OUTPUT_FORMAT` selects the format:

Format	Contents
`aout`	Today's a.out. Allowed only if the loadable sections form a text part followed by a data part, each contiguous. Otherwise the linker reports an error that suggests another format.
`binary`	A memory image from the lowest LMA to the highest, with gaps filled with the `FILL` byte. Today's `-d` produces this format.
`srec`	Motorola S-records, one run of records per loadable section, at its LMA, with the entry point in the S9 record.
`ihex`	Intel hex, likewise.

The S-record and Intel hex writers move from `objcopy01` into a small shared source file used by both programs, so their output is identical. `objcopy01` keeps working on a.out files as today.

8 Command line

Option	Meaning
`-S script`	Use the linker script in `script` instead of the default script.
`-A symbol=value`	Define a symbol, as if the script had `symbol = value;`.
`-Z`	Treat orphan input sections as an error.
`-M`	Also print the layout: each memory region and output section with its VMA, LMA and size, and the input sections in it.
`-P`	Print the default linker script and exit.

The letters are chosen to avoid `ld01`'s existing options, which already use `-T`, `-D`, `-L` and `-s`, the letters GNU `ld` uses for these.

The existing options keep their meaning. With a script:

- `-T addr` sets the address of the output section `.text`, and `-D addr` that of `.data`, overriding the script. It's an error if the script has no such section.
- `-H` adds to `__heap_top` as today.
- `-d` is the same as `OUTPUT_FORMAT(binary)`.
- `-C`, `-l`, `-L` and `-O` work as today. Their files can be matched by name in the script.

8.1 Printing the default script

`-P` prints the default script on standard output and exits with status 0. No input files are needed; any that are given are ignored. The printed script fills in the values from the other options on the command line:

- the `-T` and `-D` addresses;
- the `-H` heap size;
- `OUTPUT_FORMAT(binary)` instead of `aout` with `-d`.

The printed script is a complete, working script. Saving it and linking with `-S` gives exactly the same output as linking without a script, given the same other options:

```
$ ld01 -P -T0x434c > mc10.ld
$ ld01 -S mc10.ld -lnum -Cstart -o main.aout main.o screen.o
```

This makes it easy to start from the default layout and change only what you need. `-P` with `-S` is an error, since `-P` shows the script that `-S` replaces.

`ld01` stores the default script as text with placeholders for these values. The same text is used for linking and printed by `-P`, so the two can't differ.

With `-i` (separate instruction and data spaces), the printed script places `.data` at the `-D` address or 0 and notes in a comment that `-i` must still be given. That flag selects an a.out header format, which isn't part of the script.

9 The default script

The current layout is written as a script, compiled into `ld01`. It's used when no `-S` is given, and `ld01 -P` prints it. With no other options, `-P` prints:

```
/* Default linker script for ld01.
   Values from the command line: -T 0x0000, no -D, no -H. */

OUTPUT_FORMAT(aout)

SECTIONS
{
```

```
.text 0x0000 : SUBALIGN(4)
{
    PROVIDE(__btext = .);
    PROVIDE(__seg0DL = .);
    *(LOC0)
    PROVIDE(__seg0DH = .);
    PROVIDE(__seg0CL = .);
    *(COMMON0)
    PROVIDE(__seg0CH = .);
    PROVIDE(__seg1DL = .);
    *(LOC1)
    PROVIDE(__seg1DH = .);
    PROVIDE(__seg1CL = .);
    *(COMMON1)
    PROVIDE(__seg1CH = .);
    PROVIDE(__etext = .);
}

.data __etext : SUBALIGN(4)
{
    PROVIDE(__bdata = .);
    HIDDEN(__dend = .);

    ONEPAGE
    {
        PROVIDE(__seg2DL = .);
        HIDDEN(__dpos = .);
        *(LOC2)
        HIDDEN(__dend = . != __dpos ? . : __dend);
        PROVIDE(__seg2DH = .);
        PROVIDE(__seg2CL = .);
        *(COMMON2)
        PROVIDE(__seg2CH = .);
    }

    PROVIDE(__seg3DL = .);
    HIDDEN(__dpos = .);
    *(LOC3)
    HIDDEN(__dend = . != __dpos ? . : __dend);
    PROVIDE(__seg3DH = .);
    PROVIDE(__seg3CL = .);
    *(COMMON3)
    PROVIDE(__seg3CH = .);

    /* ... the same for LOC4 to LOC15 ... */

    PROVIDE(__end = .);
}
```

```
    PROVIDE(__edata = __dend);
    PROVIDE(__heap_top = __end + 0x0000);
    PROVIDE(__segoff = 0);
}
```

How it reproduces today's layout:

- **Text and data.** Counters 0 and 1 go into `.text` at the `-T` address. Counters 2 to 15 go into `.data`, which follows the text, or starts at the `-D` address.
- **Common blocks** follow their counter's data, as today.
- **Padding.** `SUBALIGN(4)` reproduces the 4-byte padding of each module's part of each counter.
- **Counter 2.** `ONEPAGE` reproduces its special rule.
- **`__edata`** is the end of the data part of the last data counter that isn't empty. `__dend` tracks it: after each counter, it moves to the current address if that counter added anything. The common blocks after that point count as bss in the a.out header.
- **Entry point.** The script has no `ENTRY`, so the entry point is chosen as today.
- **Linker symbols.** All the symbols `ld01` defines today, including the `__segXDL`, `__segXDH`, `__segXCL` and `__segXCH` boundaries of each counter, are `PROVIDE`d, so they only appear when used, as today. `-P` prints the full script, with every counter written out.

The first phase of the implementation must show that this script produces byte-identical output to today's `ld01` for every test (see Testing).

10 Examples

10.1 A RAM program for the MC-10

This program is loaded from cassette at `$434C`, like the demo. It gives the stack an address and puts the code of `crtstart.o` first.

```
ENTRY(__start)

SECTIONS
{
    .text 0x434C :
    {
        crtstart.o(LOC0)
        *(LOC0 LOC1)
    }
    .data :
    {
        *(LOC2 LOC3 LOC4 LOC5 LOC6 LOC7 LOC8 LOC9
          LOC10 LOC11 LOC12 LOC13 LOC14 LOC15)
        *(COMMON)
    }
```

```
    PROVIDE(__stack_top = 0x4FFF);
}

$ ld01 -S mc10.ld -lnum -o main.aout crtstart.o main.o screen.o
```

10.2 A ROM program

A program in an 8 KB EPROM at `$E000`. The vector table is in its own module, `vectors.o`, which nothing refers to, so it's kept explicitly. Initialized data runs in internal RAM but is stored in ROM after the code. The startup code copies it with the symbols the script defines.

```
ENTRY(reset)
OUTPUT_FORMAT(srec)

MEMORY
{
    ROM (rx) : ORIGIN = 0xE000, LENGTH = 0x1FF0
    VEC (r)  : ORIGIN = 0xFFF0, LENGTH = 16
    RAM (rw) : ORIGIN = 0x0080, LENGTH = 128
}

SECTIONS
{
    .text :
    {
        *(LOC0 LOC1)
    } > ROM = 0xFF

    .data :
    {
        __data_start = .;
        *(LOC3)
        __data_end = .;
    } > RAM AT> ROM

    .bss (NOLOAD) :
    {
        *(COMMON)
    } > RAM

    .vectors :
    {
        KEEP(vectors.o(LOC2))
    } > VEC

    __stack_top = ORIGIN(RAM) + LENGTH(RAM) - 1;
    __data_load = LOADADDR(.data);
```

```
    ASSERT(SIZEOF(.vectors) == 16, "vector table must be 16 bytes")
}
```

`crt0.s` copies `.data` from ROM to RAM using `__data_load`, `__data_start` and `__data_end`:

```
        IMPORT  __data_load, __data_start, __data_end, __stack_top
reset   lds     #__stack_top
        ldx     #__data_load
        ...
```

10.3 Fixed addresses

A module of entry points that other programs call at known addresses, and a memory-mapped port:

```
SECTIONS
{
    .jumptab 0xF000 : { KEEP(jumptab.o(LOC0)) }
    .text           : { *(LOC0 LOC1) }
    .data           : { *(LOC*) *(COMMON) }

    PORT1 = 0x0002;
    PORT2 = 0x0003;
}
```

11 Error checking

The linker reports:

- syntax errors, with the script's file name and line number;
- overlapping sections, both by VMA and by LMA;
- sections that don't fit their region, with how many bytes over;
- addresses above `$FFFF`, and sections that wrap around;
- `aout` output that the layout can't be expressed in;
- undefined symbols, as today, and symbols defined by both the script and an object file;
- orphans, as a warning or with `-Z` an error;
- failed `ASSERT`s, with their message;
- assignments that never settle on a value, such as `a = b + 1; b = a;`.

12 Implementation

The implementation has five parts.

12.1 1. Per-module placement

Today, placement is computed per location counter (`segbase[]`, `segsz[]`, `combase[]`, `comsz[]`, `segadj[]`). This becomes a table of *input section* records, one per loaded module and location counter, each with:

- its size, and the size of its common part;
- its VMA and LMA, and its offset in the output;
- the output section it belongs to.

`linkmod()` then uses the module's own records:

- `setseg(seg)` seeks to the record's output offset;
- offset relocations add the record's VMA instead of `segbase[seg]`;
- PC-relative relocations subtract the VMA of the current position, as today with `spos`.

Symbol values become the record's VMA plus the symbol's offset in the module, instead of being accumulated across modules.

This part changes no behavior. The fixed layout is computed into the new records with the existing rules, including the 4-byte padding.

12.2 2. Script parsing

A recursive-descent parser in a new file, `ld/script.c`, turns a script into a tree of commands, sections and expressions. The default script is a string compiled into `ld01` and parsed the same way. Scripts are small, so there's no need for a separate scanner generator.

12.3 3. Layout

After module selection, the linker:

1. Assigns each input section to an output section, using the first matching pattern, or the orphan rule.
2. Walks the sections in order, advancing `.` through each one. It applies alignment and gaps, gives each input section its VMA and LMA, and records symbol assignments.
3. Evaluates the symbol assignments. If any value changed, it repeats step 2. A limit of 10 passes catches assignments that never settle.
4. Checks for overlaps, region overflows and failed `ASSERT`s.

Module selection runs before layout. It treats `KEEP` patterns and script symbols as extra roots for `linkrefs()`.

12.4 4. Output

Writing builds a list of loadable output sections, ordered by LMA. The a.out writer checks that they fit its text-and-data model. The binary writer fills gaps, and the S-record and Intel hex writers write each section's bytes at its LMA. The symbol table is written as today for a.out.

12.5 5. Documentation

The `ld01` man page gets a section on scripts, and the user's manual gets a chapter with the examples above.

13 Testing

The project has no automated tests today. This work adds a `tests/` directory with a `make check` target:

- **Compatibility.** Link the demo program and a set of test modules without a script, and compare against reference outputs produced by today's `ld01`. These must be byte-identical after part 1 and after every later part.
- **Default script.** For several combinations of `-T`, `-D`, `-H`, `-d` and `-i`, save the output of `ld01 -P` with those options. Then link the same inputs with `-S` and that script, and with the same options. The output must equal the no-script output.
- **`-P`.** `ld01 -P` exits with status 0 without any input files, and `-P` with `-S` is an error.
- **Features.** For each script feature, a small script, test modules and the expected output: S-records, a map from `-M`, or an error message.
- **Errors.** Each check in Error checking has a test that triggers it.

14 Phases

Phase	Work	Result
1	Per-module placement; tests and reference outputs	No change in behavior, except a fix for `-T`/`-D` addresses that aren't a multiple of 4. **Done.**
2	Parser, default script, `SECTIONS`, `ONEPAGE`, symbols including `HIDDEN`, `ENTRY`, `-S`, `-A`, `-P`, `-M` layout	Scripts for single-region programs
3	`MEMORY`, regions, `AT`, `NOLOAD`, `KEEP`, `PROVIDE`, `FILL`, `ASSERT`, orphans, `-Z`	ROM/RAM layouts
4	`binary`, `srec` and `ihex` output; shared writers with `objcopy01`	Multi-region output
5	Man page, manual chapter, examples	Documented

Each phase can be merged and released on its own.

15 Open questions

1. **Option letters.** Are `-S`, `-A` and `-Z` acceptable, or should `ld01` take long options such as `--script` and `--defsym`, as GNU `ld` does?
2. **A separate bss counter.** `as01` puts `.DATA` and `.BSS` in the same location counter, and `RMB` space is written as zeros, so only common blocks are really uninitialized. Should `.BSS` get its own counter, so that scripts can mark it `NOLOAD`?
3. **Direct-page addressing.** A script can place a counter in page zero (`$0000`–`$00FF`), but `as01` still uses extended addressing for relocatable symbols, because it doesn't know their addresses. 8-bit relocations exist in the object format, so `as01` and `ld01` could support direct addressing of relocatable symbols in page zero. This would be a separate change.
4. **Module padding.** The 4-byte padding of each module's part of each counter wastes space on a 6801. The default script keeps it for compatibility. Should a future version drop it from the default script?
5. **ORG in relocatable modules.** In a relocatable module, `ORG` doesn't set an absolute address. A forward `ORG` becomes a gap from the module's start, and a backward one makes `ld01` fail with "seg 0 has wrong size". Should `as01` reject `ORG` in relocatable modules, leaving scripts as the way to place code at fixed addresses?